

The image features the text "DL for ENG" in a bold, 3D, gold-colored font. The letters are floating on a vast, blue ocean with visible ripples and small waves. In the background, there are rolling blue hills or mountains under a sky filled with soft, grey clouds. The overall scene has a serene and expansive feel.

DL for ENG

LECTURE 9.2

Turbulent Flow

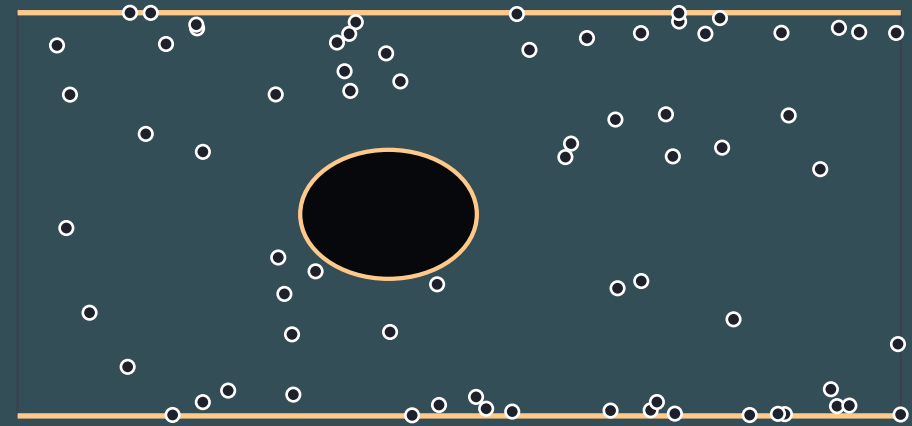
Remus Teodorescu

*With support from Research Assistant Noman Khan -
nomank@energy.aau.dk*

Aalborg University · ret@et.aau.dk

Deep Learning for Engineering · 2026

AVERAGE · CLOSE · ASSIMILATE



Ex_09.2: the mean flow, and points at the walls

Above the ceiling of the last lecture the equations are averaged, and the average is not closed: a term is left that no equation determines. A model supplies it, and then the turbulent problem is the laminar one again. That is the whole trick, and the whole weakness.

Learning Objectives

SECTIONS, IN ORDER

- 1 **Turbulence** — the range of scales, and averaging the equations
- 2 **Closure** — the term averaging leaves, and how it is modelled
- 3 **Flow with a model** — solving it, checking it, and assimilating data

Liu has no flow chapter, so the physics is Pope's and the practice is Ex_09.2's channel, where the closure is a number you choose.

READ, AND WHAT EACH ANSWERS

Pope ch. 4, 6	why can a turbulent flow not be resolved?
Pope ch. 10	what does an eddy viscosity assume?
Pope ch. 7	why is the wall the expensive part?

The Range of Scales

Turbulence — a flow that is chaotic and carries energy across a range of eddy sizes, from the width of the pipe down to the scale at which viscosity turns motion into heat. The ratio of the two grows with the Reynolds number, and so does the number of points a solver needs.

THE SMALLEST EDDY, AND WHAT IT COSTS

$$\frac{\eta}{L} \sim Re^{-3/4}$$

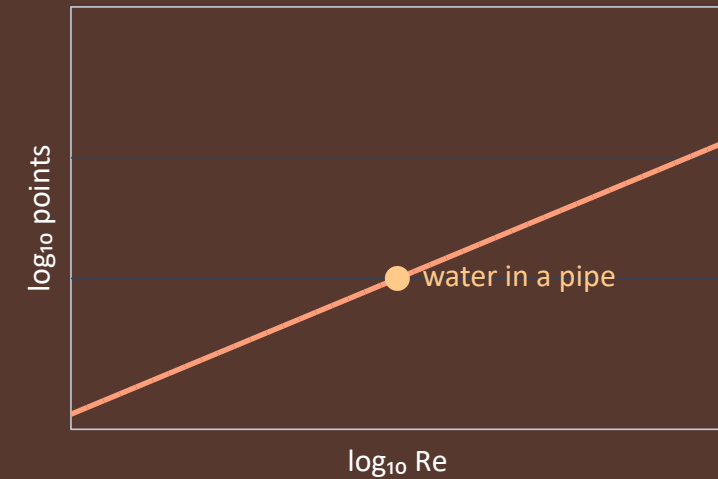
$$N \sim Re^{9/4} \text{ per dimension, } Re^3 \text{ in 3D}$$

resolve every eddy in three dimensions and the count grows as Re cubed

THE ARITHMETIC IN PYTHON

```
eta_over_L = Re ** -0.75          # Kolmogorov's smallest eddy
n_per_dim = Re ** 0.75            # points to resolve it, per axis
print(n_per_dim ** 3)            # in three dimensions
```

POINTS TO RESOLVE A 3-D FLOW



At $Re = 10^4$ that is about a billion points, and a collocation set is thousands. This is not a limit of networks: it is why turbulence modelling exists.

The Navier–Stokes Problem

Navier–Stokes problem — given a smooth initial velocity field of finite energy, do the three-dimensional incompressible equations keep a smooth solution for all time, or can the speed become unbounded in finite time? One of the seven Clay Millennium Prize problems, stated so that a smooth applied force is allowed.

THE EQUATIONS THE QUESTION IS ABOUT

$$\mathbf{u}_t + (\mathbf{u} \cdot \nabla) \mathbf{u} = -\frac{1}{\rho} \nabla p + \nu \nabla^2 \mathbf{u} + \mathbf{f}$$

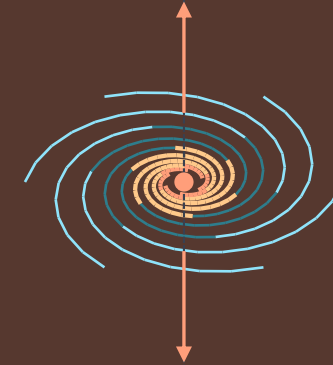
$$\nabla \cdot \mathbf{u} = 0$$

momentum, and the constraint

WHAT WAS CLOSED, AND WHAT WAS NOT

- **closed** — a breakdown with a force, machine-checked
- **open** — the unforced question, the one people mean
- **the prize** — unclaimed, and Clay still lists it open

THE FLOW, AND WHO PROVED WHAT



a vortex spiralling inward, stretched along its axis: warm where it turns fastest, and the speed is unbounded at the core

who

what they proved

Martínez-Zorúa, Córdoba	the cascade, by hand
Alpöge, Buckmaster	forced Euler, announced first
<u>OpenAI's system</u>	unforced Euler, then forced N–S
Clay Institute	has not accepted it

Reynolds Averaging

Reynolds averaging — splitting every field into a mean and a fluctuation, then averaging the equations. What comes out looks like the flow equations for the mean, with one extra term: the average of the fluctuations multiplied together, which no equation determines.

SPLIT THE FIELD, AVERAGE THE EQUATIONS

$$u_i = \bar{u}_i + u'_i, \quad \overline{u'_i} = 0$$

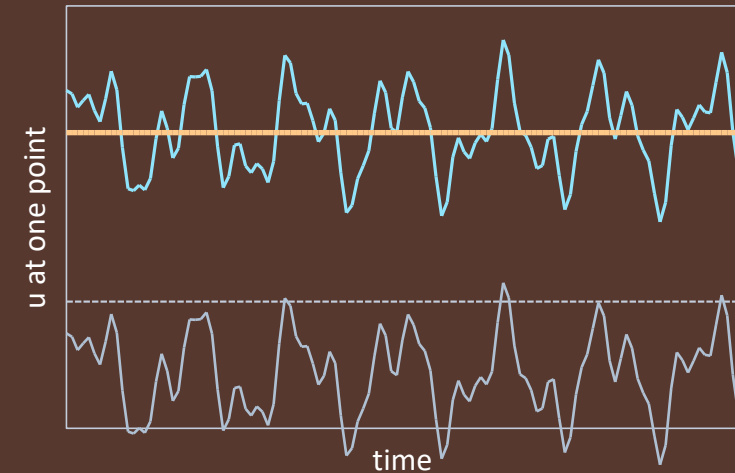
$$\bar{u}_j \partial_j \bar{u}_i = -\frac{1}{\rho} \partial_i \bar{p} + \nu \nabla^2 \bar{u}_i - \partial_j \overline{u'_i u'_j}$$

the last term is what survives the averaging, and it is not known

THE MEAN FLOW IN PYTORCH

```
psi, p = split(net(xy))                # the MEAN field, not the flow
u, v = grad(psi, y), -grad(psi, x)     # mean velocity from psi
F = conv(u, v) + grad(p, x)/rho - visc(u) + stress_term # the last one?
```

A SIGNAL, ITS MEAN, ITS FLUCTUATION



Cyan the signal at one point, amber its mean, grey the fluctuation below. The mean is what a network represents; the fluctuation carries the missing momentum.



The Closure Problem

Closure problem — the averaged equations contain the Reynolds stresses, the averaged products of the fluctuations, and there are more unknowns than equations. Writing an equation for them produces new unknown averages, so at some point a model must be supplied from outside.

MORE UNKNOWN THAN EQUATIONS

$$\tau_{ij} = -\rho \overline{u'_i u'_j}$$

six unknowns in three dimensions, and no equation that closes them

- **not a numerical problem** — it is in the equations
- **every solver** — CFD or PINN — needs the same model

TALK • 2 MIN How would you explain, in your own words, why averaging an equation can lose information? Then, in the card on the right, which row would you trust least on a flow with separation, and what would you do about it?

WHAT CLOSES IT, AND AT WHAT PRICE

model	supplies	costs
constant ν_t	one number	crudest
mixing length	a profile	tuned per case
two-equation	k and ϵ	two more PDEs
learned ν_t	a network	needs data

Ex_09.2 uses the first: one number for the whole channel.

THE MISSING TERM

```
F = conv(u, v) + grad(p, x)/rho - nu*lap(u)
F = F + stress_term           # unknown: supply a model
```

Eddy Viscosity

Eddy viscosity — the assumption that the Reynolds stresses act like a viscosity: proportional to the mean strain rate, through a coefficient ν_t . The averaged equations then become the laminar equations with ν replaced by $\nu + \nu_t$, and the solver of the last lecture applies again.

A MODEL THAT LOOKS LIKE VISCOSITY

$$-\overline{u'_i u'_j} = \nu_t (\partial_j \bar{u}_i + \partial_i \bar{u}_j)$$

$$\nu_{\text{eff}} = \nu + \nu_t$$

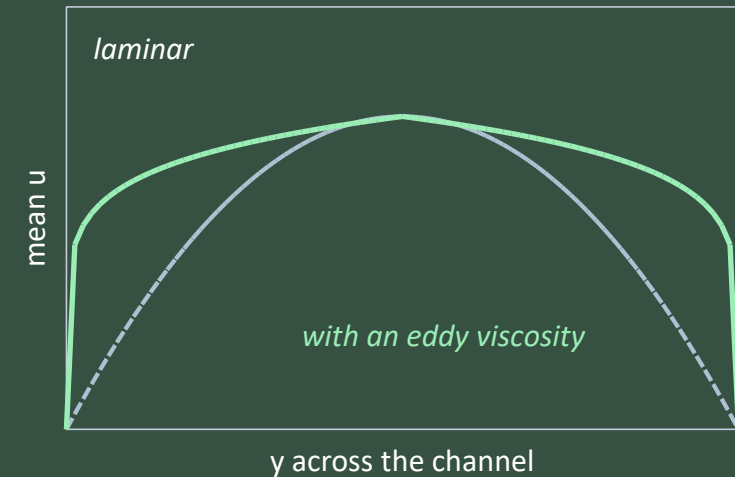
prescribe ν_t and the turbulent problem is the laminar one again

- **what it assumes** — stresses aligned with the strain
- **where it fails** — separation, swirl, strong curvature

THE CLOSURE IN PYTORCH

```
nu_eff = nu + nu_t                # one number for the channel
F = conv(u, v) + grad(p,x)/rho - nu_eff * lap(u)  # the laminar residual
```

SCHEMATIC · THE MEAN PROFILE IT GIVES



Schematic: the model flattens the middle and steepens the wall. That steeper wall gradient is where the drag comes from, and where the points must be.

The Wall

Law of the wall — next to a wall the mean velocity follows a fixed shape when measured in wall units: y^+ for the distance and u^+ for the speed. Resolving that layer costs most of the points in any turbulent calculation, which is why a wall function replaces it with the formula.

WALL UNITS, AND WHAT THEY COST

$$y^+ = \frac{u_\tau y}{\nu}, \quad u_\tau = \sqrt{\frac{\tau_w}{\rho}}$$

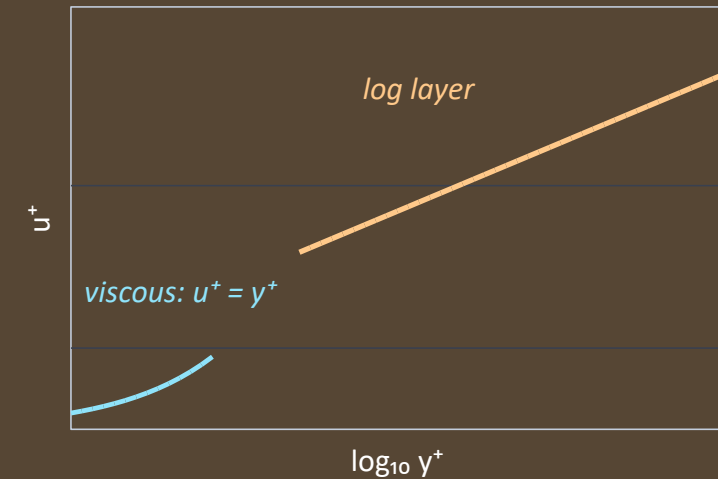
$$u^+ = \frac{1}{\kappa} \ln y^+ + B, \quad \kappa \approx 0.41, B \approx 5.0$$

the log layer: the same two constants for every wall-bounded flow

NEAR-WALL POINTS IN PYTORCH

```
b = torch.rand(n, 1) # uniform in [0, 1)
y = torch.where(b < 0.5, 2*b*b, 1 - 2*(1-b)**2) # graded to both walls
pts_f = torch.cat([xy_interior, near_wall_cloud]) # residual points, not BC
```

THE LAW OF THE WALL



Ex_09.2 resolves the layer with graded points instead, because its Reynolds numbers are modest. Notebook 03 turns the grading off and watches the drag.

RANS with a PINN

RANS with a network — the averaged equations solved by the loop of L7, with the closure supplied: the network outputs the mean stream function and pressure, ν_{eff} carries the model, and the residual is the one of L9.1 with a larger viscosity.

THE LAMINAR SOLVER, WITH A MODEL IN IT

$$\nu_t = \mathcal{N}_{\psi}^t(\mathbf{x}) \geq 0$$

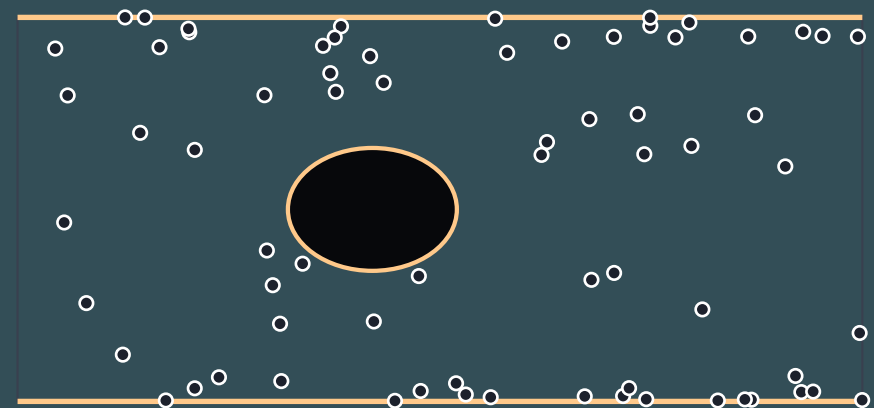
or a second network for ν_t , constrained non-negative

- **unchanged from L9.1** — ψ , no-slip, the outlet, the sampling
- **changed** — one coefficient, and what it stands for

RANS IN PYTORCH

```
psi, p = split(net(xy))           # the mean field
nu_eff = nu + softplus(nu_t_raw)  # the closure, non-negative
loss = F.pow(2).mean() + w_bc * L_bc  # L9.1's loss, unchanged
```

THE CHANNEL OF EX_09.2



points graded to the walls

you choose	the obstacle's shape and size
the closure	none, or one uniform ν_t
you report	drag, and the pressure drop

Prescribe or Learn

Prescribed or learned closure — ν_t can be fixed before training, which is explicit and crude, or represented by a second network and learned, which needs data to identify it. A learned ν_t must be kept non-negative, because a negative effective viscosity makes the residual unstable.

A CHOICE, AND IT IS NOT FREE EITHER WAY

- **prescribed** — one number, stated in the report
- **learned** — a field, and it needs measurements
- **either way** — the answer inherits the model

TALK · 2 MIN How would you decide whether to fix a model coefficient or learn it, in your own words? Then, in the table on the right, which row would you defend to a client, and what would you have to show them?

THE TWO ROUTES

	prescribed	learned
ν_t is	a number you pick	a network's output
needs	a judgement	measurements
report	the number	the data and the fit
risk	wrong everywhere	right only where data is

Ex_09.2 prescribes it, and asks you to say in the report which of the two your own case would need. That sentence is what the exercise is marked on.

Data Assimilation

Data assimilation — fitting a flow field to sparse measurements while the averaged equations hold as a constraint. The network fills in between the sensors, recovers quantities that were never measured, such as the pressure, and needs no closure where the data speaks.

THE USE THAT A SOLVER CANNOT COPY

$$\mathcal{L} = \mathcal{L}_{RANS} + w_d \mathcal{L}_{data}$$

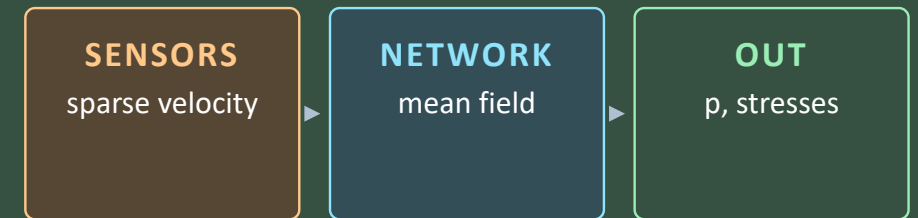
the equations as a constraint, the measurements as the target

- **sparse velocity in** — pressure and stresses out
- **no interpolation** — the equations do the filling

ASSIMILATION IN PYTORCH

```
L_data = (u(x_s) - u_meas).pow(2).mean() # what the sensors saw
loss = L_rans + w_d * L_data             # physics, and measurement
p_field = p(xy_grid)                     # never measured, still recovered
```

WHAT COMES IN, AND WHAT COMES OUT



a CFD code cannot take the data as input

a PINN takes it as one more loss term

This is the case where the method is clearly the right tool, and it is the one to put in a proposal.



What to Report

Reporting a turbulent solve — an engineering quantity with the model that produced it: the drag coefficient and the pressure drop, the closure and its value, the near-wall sampling, and the check that the mean flow is symmetric where it should be. A loss value is not a result.

THE NUMBERS, AND WHAT MADE THEM

$$C_D = \frac{F_D}{\frac{1}{2}\rho U^2 D}, \quad \Delta p = \bar{p}_{in} - \bar{p}_{out}$$

drag from pressure and shear on the obstacle; the drop across the channel

- **quote with it** — the closure, and the wall sampling
- **the smooth network** — biased towards a smooth answer

THE REPORT IN PYTORCH

```
C_D = drag(model, obstacle) / (0.5 * rho * U**2 * D) # the number asked for
dp = p(inlet).mean() - p(outlet).mean()             # the pressure drop
sym = (u(xy) - u(mirror(xy))).abs().max()           # symmetry, a free check
```

WHAT A NUMBER NEEDS BESIDE IT

report	without it
C_D and Δp	the result is unreadable
the closure	the result is unrepeatable
the wall points	the drag is unexplained
a symmetry check	a wrong field passes

Notebook 03 compares five shapes at matched blockage. Similar losses there do not mean equally right answers, and the sharp corners are where the smooth network shows its bias.

Advanced Topics

RELATED TOPICS LEFT OUT, AND WHERE THEY MATTER

topic	what it adds	read	where it makes a difference
two-equation models	k and ϵ , transported	Pope ch. 10	industrial CFD
wall functions	the log law instead of points	Pope ch. 7	high Reynolds number
large-eddy simulation	resolve the large, model the small	Pope ch. 13	unsteady loads
<u>diffusion surrogates</u>	a sampler guided by the residual	UDL ch. 14 – 18	one model, many conditions
learned closures	a model trained on data	Pope ch. 10	families of similar flows

WHY THEY ARE LEFT OUT

Ex_09.2 runs at modest Reynolds numbers with one prescribed coefficient, which is enough to show what a closure does and what it costs. The last row is the open research question that the mini-project in L13 takes up.

Exercise 9.2

THE NOTEBOOKS, IN ORDER

- **00** — the channel, your obstacle, and the sampling
- **01** — the mean-flow residual, written by you
- **02** — the control panel: Reynolds number and closure
- **03** — five shapes, compared at matched blockage
- **04** — compares them, and the report

THE QUESTION EACH NOTEBOOK ANSWERS

01 does the mean flow look right?

02 what does the closure change?

03 which shape has the least drag?

REPORT QUESTION, TO CONCLUDE

Your drag coefficients came from a closure you chose rather than computed. Say what they are worth, what measurement would make them worth more, and whether your own case needs a prescribed or a learned closure.

Key Takeaways

TURBULENCE

what it is	a range of scales, growing with Re
function	the mean is what can be solved for
train, tune	nothing: the closure is a choice
vs laminar	averaging leaves an unknown term
used for	every real flow in engineering

FLOW WITH A MODEL

what it is	the laminar solver with $v + v_t$
function	the mean field, drag, pressure drop
train, tune	points at the wall; v_t prescribed
vs a resolved solve	the answer inherits the model
used for	assimilating data, not out-solving CFD

Next — L10.1: the same method on a battery, where the unknowns are chemistry.

Questions

1 – 5 · TURBULENCE

- 1 What makes a turbulent flow hard to compute: what is the range of scales, and how does the point count grow with Re ? *Pope ch. 6, 9; Ex_09.2 notebook 02*
- 2 What does Reynolds averaging do to the equations, and which term survives it? *Pope ch. 4; Ex_09.2 notebook 01*
- 3 What is the closure problem, and why can it not be removed by adding another equation? *Pope ch. 4.4; Ex_09.2 notebook 01*
- 4 How does a turbulent mean flow differ from the laminar flow of L9.1, and what does that cost you? *Pope ch. 4, 10*
- 5 What does an eddy viscosity assume, and what does $v + v_t$ turn the problem into? *Pope ch. 10; Ex_09.2 notebooks 02 – 03*

6 – 10 · FLOW WITH A MODEL

- 6 What does a RANS network solve for, and where does the closure enter the residual? *Ex_09.2 notebook 01*
- 7 Why is the wall the expensive part, what is y^+ , and what does a wall function replace? *Pope ch. 7; Ex_09.2 notebooks 01, 03*
- 8 When would you prescribe v_t and when learn it, and why must a learned one stay non-negative? *Ex_09.2 notebooks 02 – 03*
- 9 Why is a smooth network biased on a turbulent flow, and where would that bias show on a sharp-cornered obstacle? *Ex_09.2 notebooks 01 – 03*
- 10 What is data assimilation, and why is it the honest use of the method here rather than out-solving a CFD code? *Ex_09.2 notebooks 02 – 03*

NEXT

L10.1 — Batteries and Heat

The same loop again, where the unknowns are a material's chemistry and the measurements are a voltage curve.

BEFORE THE NEXT SESSION

read	Pope ch. 4, 7, 10
run	Ex_09.2 notebooks 00 to 03
bring	your drag, and the closure you used
think about	which results a client could use

Block nine is finished: a flow that can be resolved, and one that cannot.